Introduction to Machine Learning

Session 6 Lecture Notes

Kernel Methods I

Kernels — a different way of measuring similarity of two cases. Instead of distance, we measure similarity.

$$K(x, y) = \varphi(x) \cdot \varphi(y)$$ *Kernel trick*

$\varphi(x)$ — function lifting x vector to a higher dimension space in a non-linear way.

We can derive the algebra for the kernel, so that we can bypass the 'lifting' and 'dot product' by just calculating the kernel result.

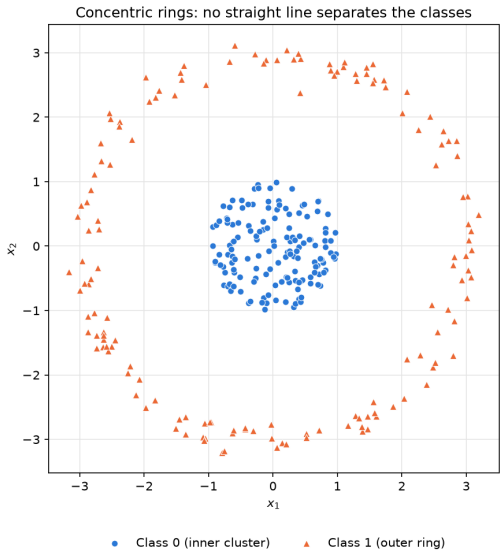
The function is a valid kernel if it produces a Gram matrix from $K = \Phi\Phi^{\mathrm{T}}$ that is:

- symmetric,
- eigenvalues ≥ 0 .

The Cover's Theorem

A complex pattern classification problem, which is nonlinearly separable in a low dimension space, is much more likely to become linearly separable when projected into a higher dimensioned feature space via non-linear transformation.

3D lift Kernel example

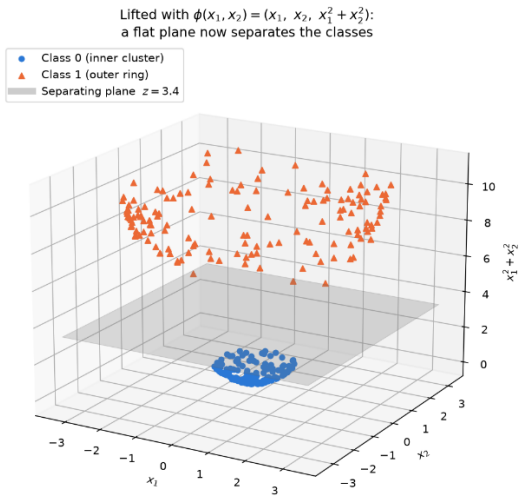


$$x = [x_1, x_2]$$
$$\varphi(x) = [x_1, x_2, {x_1}^2 + {x_2}^2]$$

Data points:

$$y = [y_1, y_2]$$
$$z = [z_1, z_2]$$

$$K(y, z) = [y_1, y_2, {y_1}^2 + {y_2}^2] \cdot [z_1, z_2, {z_1}^2 + {z_2}^2] = y \cdot z \, \|y\|^2 \, \|z\|^2$$



Polynomial kernels

$$K(x,z) = (x \cdot z + c)^d = \sum_{k=0}^{d} \binom{d}{k} c^{d-k} (x \cdot z)^k$$

d — degree

$c \geq 0$ — offset/bias

For odd d, K ranges from -∞ to ∞. For even d, K ranges from 0 to ∞.

RBF Kernel (Gaussian)

Radial Basis Function kernel

$$K(x,z) = e^{-\gamma \|x-z\|^2}$$

$$\gamma = \frac{1}{2\sigma^2}$$

$$K(x,z) = e^{-\gamma \|x-z\|^2} = e^{-\gamma \|x\|^2} e^{-\gamma \|z\|^2} e^{2\gamma x \cdot z}$$

$e^{2\gamma x \cdot z} = \sum_{k=0}^{\infty} \frac{(2\gamma)^k}{k!} (x \cdot z)^k$ *Which means infinite dimensions of lift.*

Kernel Perceptron

The Perceptron

$$z = wx + b \rightarrow \begin{cases} \hat{y} = 1, & z \geq 0 \\ \hat{y} = 0, & z < 0 \end{cases}$$

$$w_{new} = w_{old} + \eta \sum_{i=1}^{n} (y_i - \hat{y}_i) x_i \qquad b_{new} = b_{old} + \eta \sum_{i=1}^{n} (y_i - \hat{y}_i)$$

$\eta = 1$

$$w = \sum_{i=1}^{n} \alpha_i (y_i - \hat{y}_i) x_i \qquad b = \sum_{i=1}^{n} \alpha_i (y_i - \hat{y}_i)$$

$w = \Sigma(i=1 \text{ to } n)\ \eta\alpha_i(y_i - \hat{y}_i)x_i$ $b = \Sigma(i=1 \text{ to } n)\ \eta\alpha_i(y_i - \hat{y}_i)$

α_i — *how many times the point has been triggered*

If we substitute y_i from class (1, 0) to (1, −1), this can be simplified to:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i \qquad b = \sum_{i=1}^{n} \alpha_i y_i$$

Then the decision boundary:

$$f(x) = w \cdot x + b = \Sigma \sum_{i=1}^{n} \alpha_i y_i (x_i \cdot x) + \sum_{i=1}^{n} \alpha_i y_i$$

This can be expanded with different kernels - Kernel Perceptron

$$f(x) = w \cdot x + b = \sum_{i=1}^{n} \alpha_i y_i \boldsymbol{K}(\boldsymbol{x}_i, \boldsymbol{x}) + \sum_{i=1}^{n} \alpha_i y_i$$

In training we compute α values per point; with each mistake the boundary is calculated based on:

$$f(x) = w \cdot x + b = \sum_{i=1}^{n} \alpha_i y_i \boldsymbol{K}(\boldsymbol{x}_i, \boldsymbol{x}) + \sum_{i=1}^{n} \alpha_i y_i$$

After training, **for polynomial kernels**, you can optionally collapse the result into w:

$$w = \sum_{i=1}^{n} \alpha_i y_i \varphi(x_i) \qquad b = \sum_{i=1}^{n} \alpha_i y_i$$

Then the prediction can be:

$$f(x) = w \cdot \varphi(x) + b$$

For RBF kernel,

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i \boldsymbol{K}(\boldsymbol{x}_i, \boldsymbol{x}) + \sum_{i=1}^{n} \alpha_i y_i$$

We can't really calculate $\varphi(x)$, as it's infinite dimensional, so for the processing it behaves a lot like KNN → the whole training data set needs to be remembered.